

Contents

1	Problem 1	2
2	Problem 2	2
3	Problem 3	2
4	Problem 4	3
5	Problem 5	3
5.1	Part 1:	3
5.2	Part 2:	4
6	Problem 6	4
7	Problem 7	5
8	Extra Credit (20 Pts)	7
8.1	Plot generated by code:	8
8.2	Code for the extra credit:	14

ECE 661 Homework 1

Name: Syed Ali A. Shah
Email shah634@purdue.edu

August 2024

Introduction:

ECE661 Fall 2024: Homework 1

TA: Rahul Deshmukh (deshmuk5@purdue.edu)

Due Date: Midnight, 28 Aug 2024

Late submissions will be accepted with penalty: -10 points per-late-day, up to 5 days.

Turn in typed solutions via BrightSpace. Additional instructions can be found at BrightSpace.

The following notation conventions are used for representing vector, matrix, and scalar variables. Boldface lowercase letters are used to represent vectors and boldface uppercase letters are used to represent matrices. Lowercase letters (without any special typeface) are used to represent scalars.

1 Problem 1

Answer 1): All the points in representational space $\mathbb{R}^3$ that are the homogeneous coordinates of the origin in the physical space are:

$$k \cdot \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix}$$

where $k \in \mathbb{R}$ and $k \neq 0$

2 Problem 2

Answer 2): No, all the points will **NOT** be the same at infinity in $\mathbb{R}^2$. Physical points $(x, y) \in \mathbb{R}^2$ whose homogeneous coordinates(HC), are of the form $\begin{pmatrix} u \\ v \\ w \end{pmatrix}$ assuming $w \neq 0$. Hence $x = \frac{u}{w}$ and $y = \frac{v}{w}$. As $w \rightarrow 0$, both x and y will approach infinity from different directions while moving away from the origin in $\mathbb{R}^2$. This direction depends on the values of u and v . Since they are the same line but approaching at different directions on the line, this implies **No**, all the points will **NOT** be the same at infinity in $\mathbb{R}^2$.

3 Problem 3

Answer 3): The conic equation is $X^T C X = 0$. A degenerate conic is given by $C = lm^T + ml^T$ and both terms are outer product of the two vectors. These outer products of vectors create a matrix whose columns are a linearly dependent, so only a matrix with a rank of 0 or 1 can be created. This means there is only one independent column vector, if the column vector of each of the 2 outer products are different then the terms will have two independent column vectors(at most). Mathematically this is:

$$\text{Rank}(lm^T + ml^T) \leq \text{Rank}(lm^T) + \text{Rank}(ml^T)$$

$$\text{Rank}(lm^T + ml^T) \leq 1 + 1$$

$$\text{Rank}(lm^T + ml^T) \leq 2$$

Overall, as each of the two terms is a vector outer product, with each having a rank 1, the rank of C must be less than or equal to 2. Hence, the matrix rank of a degenerate conic can never exceed 2.

4 Problem 4

Conic:

A conic in $\mathbb{R}^2$ is defined by a quadratic equation in the form:

$$ax^2 + bxy + cy^2 + dx + ey + f = 0,$$

where a, b, c, d, e , and f are constants. This equation can be represented as:

$$\mathbf{X}^T \mathbf{C} \mathbf{X} = 0,$$

where $\mathbf{X} = \begin{pmatrix} x \\ y \\ 1 \end{pmatrix}$ and $\mathbf{C}$ is a symmetric 3×3 matrix:

$$\mathbf{C} = \begin{pmatrix} a & \frac{b}{2} & \frac{d}{2} \\ \frac{b}{2} & c & \frac{e}{2} \\ \frac{d}{2} & \frac{e}{2} & f \end{pmatrix}.$$

This conic equation involves six coefficients (a, b, c, d, e, f) and each point on the conic provides a linear equation in these coefficients. Since the equation is homogeneous, (meaning it can be multiplied by a non-zero scalar without changing the conic it represents) five points are needed to form a conic in $\mathbb{R}^2$ because only five of these coefficients are independent. Hence, **five points** are required to define a conic, assuming no three of these points are collinear.

Degenerate Conic:

A degenerate conic in $\mathbb{R}^2$ doesn't form a standard curve like an ellipse, parabola, or hyperbola. This means other geometric shapes like a pair of intersecting lines, and parallel lines. So an equation for a degenerate conic is

$$(ax + by + c)(dx + ey + f) = 0$$

which still forms a quadratic equation. This means we have 2 points on the first line and 2 points on the second, however the two lines intersect sharing a point hence, only 3 points are sufficient to form a degenerate conic.

In conclusion, **five points** are required to define a conic in $\mathbb{R}^2$, while **three points** are sufficient to define a degenerate conic in $\mathbb{R}^2$.

5 Problem 5

5.1 Part 1:

$$l_1 = \begin{pmatrix} -4 \\ 3 \\ 0 \end{pmatrix}, l_2 = \begin{pmatrix} 5 \\ 5/2 \\ -5 \end{pmatrix}$$

$$\begin{aligned} \text{Step 1: } P_1 &= \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix}, P_2 = \begin{pmatrix} 3 \\ 4 \\ 1 \end{pmatrix} \\ \rightarrow l_1 &= P_1 \times P_2 = \begin{pmatrix} -4 \\ 3 \\ 0 \end{pmatrix} \end{aligned}$$

Step 2: $P_3 = \begin{pmatrix} 3/2 \\ -1 \\ 1 \end{pmatrix}, P_4 = \begin{pmatrix} -1 \\ 4 \\ 1 \end{pmatrix} \rightarrow l_2 = P_3 \times P_4 = \begin{pmatrix} 5 \\ 5/2 \\ -5 \end{pmatrix}$

Step 3: $l_1 \times l_2 = \begin{pmatrix} -15 \\ -20 \\ -25 \end{pmatrix} \Rightarrow \begin{pmatrix} 3 \\ 5 \\ 4 \\ 5 \\ 1 \end{pmatrix}$

The line passing through the points $(-1, 4)$ and $(\frac{3}{2}, -1)$ is: $\begin{pmatrix} 3 \\ 5 \\ 4 \\ 5 \\ 1 \end{pmatrix}$

5.2 Part 2:

Step 1: l_1 passes through $\begin{pmatrix} -1 \\ 2 \\ 1 \end{pmatrix}$ and $\begin{pmatrix} 1 \\ -2 \\ 1 \end{pmatrix} \rightarrow l_1 = \begin{pmatrix} 4 \\ 2 \\ 0 \end{pmatrix}$

Step 2: $l_1 \times l_2 = \begin{pmatrix} -10 \\ 20 \\ 0 \end{pmatrix}$

The line passing through l_2 and l_1 if l_1 is defined using the points $(1, 2)$ and $(1, 2)$ is: $\begin{pmatrix} -10 \\ 20 \\ 0 \end{pmatrix}$

This takes 2 steps as we have already computed one of the lines.

6 Problem 6

As we know the conic C is an ellipse centered at the coordinates $(1, 4)$ and $a = 1$ and $b = 2$ is the lengths of the semi-minor and semi-major axes, and the minor axis is parallel to the x-axis and the major axis is parallel to the y-axis, the general equation for the ellipse centered at $(0, 0)$ is:

$$\frac{x^2}{a^2} + \frac{y^2}{b^2} = 1$$

Then with the center at $(1, 4)$ the equation is:

$$\begin{aligned} \frac{(x-1)^2}{1^2} + \frac{(y-4)^2}{2^2} &= 1 \\ x^2 + 1 - 2x + \frac{y^2 + 16 - 8y}{4} &= 1 \\ x^2 + 1 - 2x + \frac{y^2}{4} + 4 - 2y &= 1 \\ x^2 + 0 * xy + \frac{y^2}{4} - 2x - 2y + 4 &= 0 \end{aligned}$$

General equation of a conic is:

$$Ax^2 + Bxy + Cy^2 + Dx + Ey + F = 0$$

or

$$\begin{pmatrix} x & y & 1 \end{pmatrix} \begin{pmatrix} A & \frac{B}{2} & \frac{D}{2} \\ \frac{B}{2} & C & \frac{E}{2} \\ \frac{D}{2} & \frac{E}{2} & F \end{pmatrix} \begin{pmatrix} x \\ y \\ 1 \end{pmatrix} = 0$$

Hence,

$$C = \begin{pmatrix} 1 & 0 & -2 \\ 0 & 1 & -2 \\ -2 & -2 & 4 \end{pmatrix} \rightarrow \begin{pmatrix} 1 & 0 & -1 \\ 0 & \frac{1}{4} & -1 \\ -1 & -1 & 4 \end{pmatrix}$$

Using this information, $l = Cp \rightarrow \begin{pmatrix} 1 & 0 & -1 \\ 0 & \frac{1}{4} & -1 \\ -1 & -1 & 4 \end{pmatrix} \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix} = \begin{pmatrix} -1 \\ -1 \\ 4 \end{pmatrix}$

The equation is: $-x - y + 4 = 0$

For x-axis:

$$y = 0$$

$$-x - 0 = -4 \rightarrow x = 4$$

The point of intersection of the polar line with the x-axis is: **(4, 0)**

For y-axis:

$$x = 0$$

$$-y = -4 \rightarrow y = 4$$

The point of intersection of the polar line with the y-axis is: **(0, 4)**

7 Problem 7

How: Given the vertices of the triangle v_1, v_2, v_3 , and the intersection points P_{12}, P_{23} , and P_{31} , the following steps are performed to check if the intersection points lie within the corresponding line segments of the triangle.

Let $v_1 = \begin{pmatrix} x_1 \\ y_1 \\ w_1 \end{pmatrix}$, $v_2 = \begin{pmatrix} x_2 \\ y_2 \\ w_2 \end{pmatrix}$, and $v_3 = \begin{pmatrix} x_3 \\ y_3 \\ w_3 \end{pmatrix}$ be the vertices of the triangle in homogeneous coordinates.

Assume the line l is defined by:

$$l = \begin{pmatrix} a \\ b \\ c \end{pmatrix}.$$

The intersection points P_{12}, P_{23} , and P_{31} are calculated as:

$$P_{12} = l \times L_{12}, \quad P_{23} = l \times L_{23}, \quad P_{31} = l \times L_{31},$$

where $L_{12} = v_1 \times v_2$, $L_{23} = v_2 \times v_3$, and $L_{31} = v_3 \times v_1$ are the line equations corresponding to the edges of the triangle.

To determine if each intersection point lies within the corresponding line segment, the intersection point P (for instance, P_{12}) is normalized to Cartesian coordinates:

$$\text{If } P_{12} = \begin{pmatrix} x_p \\ y_p \\ w_p \end{pmatrix} \text{ with } w_p \neq 0, \text{ then } P_{12} = \begin{pmatrix} \frac{x_p}{w_p} \\ \frac{y_p}{w_p} \\ 1 \end{pmatrix}.$$

The point P_{12} lies within the segment defined by v_1 and v_2 if:

$$\min(x_1, x_2) \leq \frac{x_p}{w_p} \leq \max(x_1, x_2) \quad \text{and} \quad \min(y_1, y_2) \leq \frac{y_p}{w_p} \leq \max(y_1, y_2).$$

Similarly, the other intersection points P_{23} and P_{31} are checked against their respective segments v_2v_3 and v_3v_1 .

(I have given a second method for checking if there is a hit or miss, below as well in case this one is not accepted)

Solving Figure 1:

$$v_1 = \begin{pmatrix} 3 \\ 5 \\ 1 \end{pmatrix}, \quad v_2 = \begin{pmatrix} 7 \\ 5 \\ 1 \end{pmatrix}, \quad v_3 = \begin{pmatrix} 5 \\ 3 \\ 1 \end{pmatrix}, \quad \alpha = 45^\circ, \quad l = \begin{pmatrix} \cos \alpha \\ \sin \alpha \\ 0 \end{pmatrix} \times \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix} = \begin{pmatrix} \sin \alpha \\ -\cos \alpha \\ 0 \end{pmatrix},$$

$$\mathbf{L}_{12} = \mathbf{v}_1 \times \mathbf{v}_2 = \begin{pmatrix} 0 \\ 4 \\ -20 \end{pmatrix}$$

$$\mathbf{L}_{23} = \mathbf{v}_2 \times \mathbf{v}_3 = \begin{pmatrix} 2 \\ -2 \\ -4 \end{pmatrix}$$

$$\mathbf{L}_{31} = \mathbf{v}_3 \times \mathbf{v}_1 = \begin{pmatrix} -2 \\ -2 \\ 16 \end{pmatrix}$$

$$P_{12} = l \times \mathbf{L}_{12} = \begin{pmatrix} 10\sqrt{2} \\ 10\sqrt{2} \\ 2\sqrt{2} \end{pmatrix} \rightarrow \begin{pmatrix} 5 \\ 5 \\ 1 \end{pmatrix}$$

$$P_{23} = l \times \mathbf{L}_{23} = \begin{pmatrix} 2\sqrt{2} \\ 2\sqrt{2} \\ 0 \end{pmatrix}$$

$$P_{31} = l \times \mathbf{L}_{31} = \begin{pmatrix} -8\sqrt{2} \\ -8\sqrt{2} \\ -2\sqrt{2} \end{pmatrix} \rightarrow \begin{pmatrix} 4 \\ 4 \\ 1 \end{pmatrix}$$

Now we need to check if any of these points is within the triangle, Two methods are given below for checking if it is in the triangle or not.

Method 1 (Checking the bounds by checking if it is between the lines formed by the triangle (v_1, v_2, v_3)):

For P_{12} : The point P_{12} lies within the segment defined by v_1 and v_2 if:

$$\min(3, 7) \leq \frac{5}{1} \leq \max(3, 7) \quad \text{and} \quad \min(5, 5) \leq \frac{5}{1} \leq \max(5, 5).$$

$$3 \leq 5 \leq 7 \quad \text{and} \quad 5 \leq 5 \leq 5$$

$$True \quad \text{and} \quad True$$

This is clearly within the triangle meaning the user HIT the triangle.

Similarly, the other intersection points P_{23} and P_{31} are checked against their respective segments v_2v_3 and v_3v_1 . Note: **For** P_{23} cannot be done in Cartesian coordinates, as $w_p = 0$

Method 2 (suggested by the GTA is given is below): Given a triangle with vertices $V_1 = (x_1, y_1)$, $V_2 = (x_2, y_2)$, and $V_3 = (x_3, y_3)$, the barycentric coordinates of a point P are computed as follows:

$$\begin{aligned} \lambda_1 &= \frac{(y_2 - y_3)(x - x_3) + (x_3 - x_2)(y - y_3)}{(y_2 - y_3)(x_1 - x_3) + (x_3 - x_2)(y_1 - y_3)} \\ \lambda_2 &= \frac{(y_3 - y_1)(x - x_3) + (x_1 - x_3)(y - y_3)}{(y_2 - y_3)(x_1 - x_3) + (x_3 - x_2)(y_1 - y_3)} \\ \lambda_3 &= 1 - \lambda_1 - \lambda_2 \end{aligned}$$

Inside the Triangle The point $P(x, y)$ is inside the triangle if:

$$\lambda_1 > 0, \quad \lambda_2 > 0, \quad \lambda_3 > 0$$

On an Edge The point $P(x, y)$ lies on an edge of the triangle if one of the λ values is 0, and the other two are positive.

Outside the Triangle The point $P(x, y)$ is outside the triangle if any of the λ values is less than 0:

$$\lambda < 0$$

The point P is inside the triangle if and only if all λ_1, λ_2 , and λ_3 are positive and sum to 1. $V_1 = (3, 5)$, $V_2 = (7, 5)$, and $V_3 = (5, 3)$, For P_1 : $\lambda_1 = \frac{1}{2}$
 $\lambda_2 = \frac{1}{2}$ $\lambda_3 = 1 - 0 - 1 = 0$

Hence: In the case of figure 1, the user aim is correct, the user does hit the triangle.

8 Extra Credit (20 Pts)

Plots are given here and the code is given below the plots (apologies for the lengthy code, tried to make it pretty and implement more than one method for checking. The extra credit was Fun! Thank you) If the text or plots are hard to read, please zoom in.

8.1 Plot generated by code:

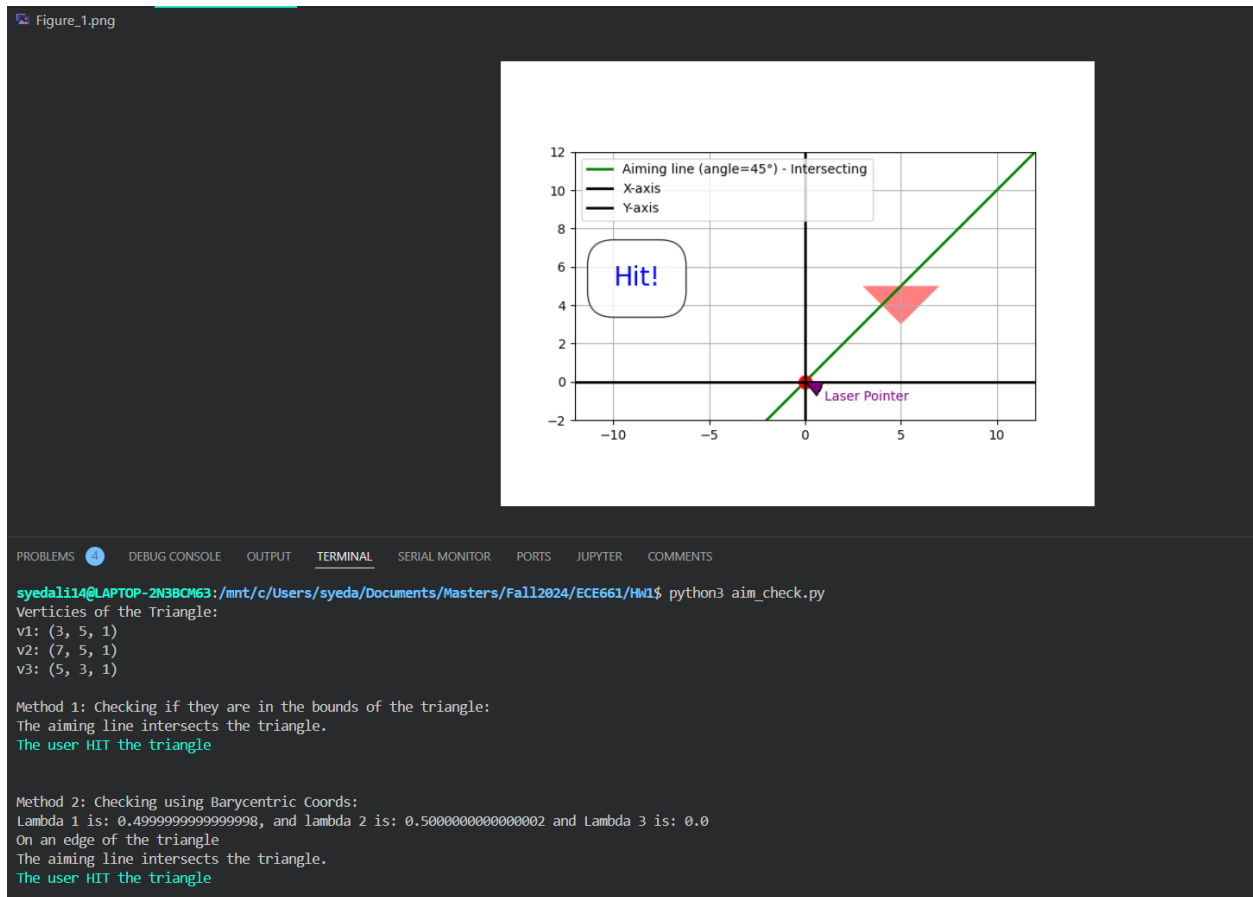


Figure 1: Figure 1

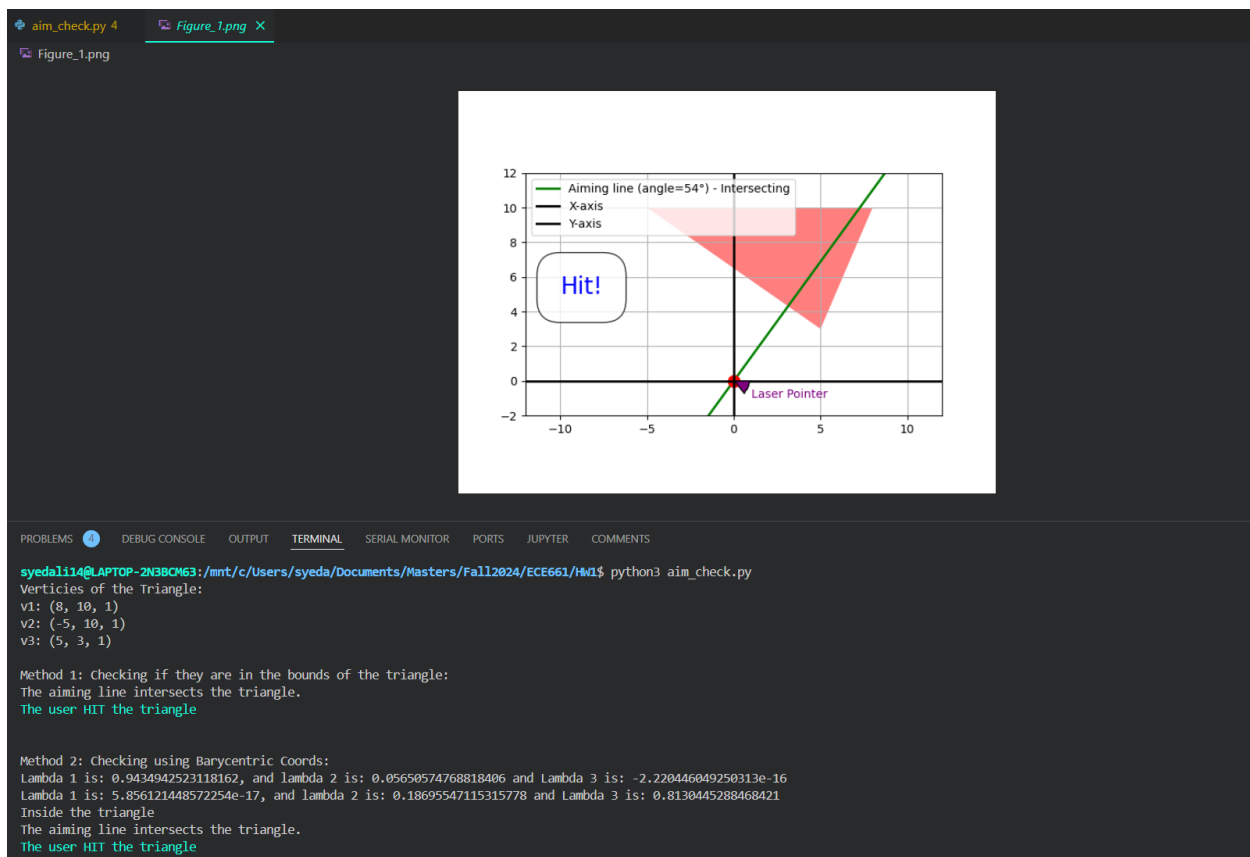


Figure 2: Figure 2

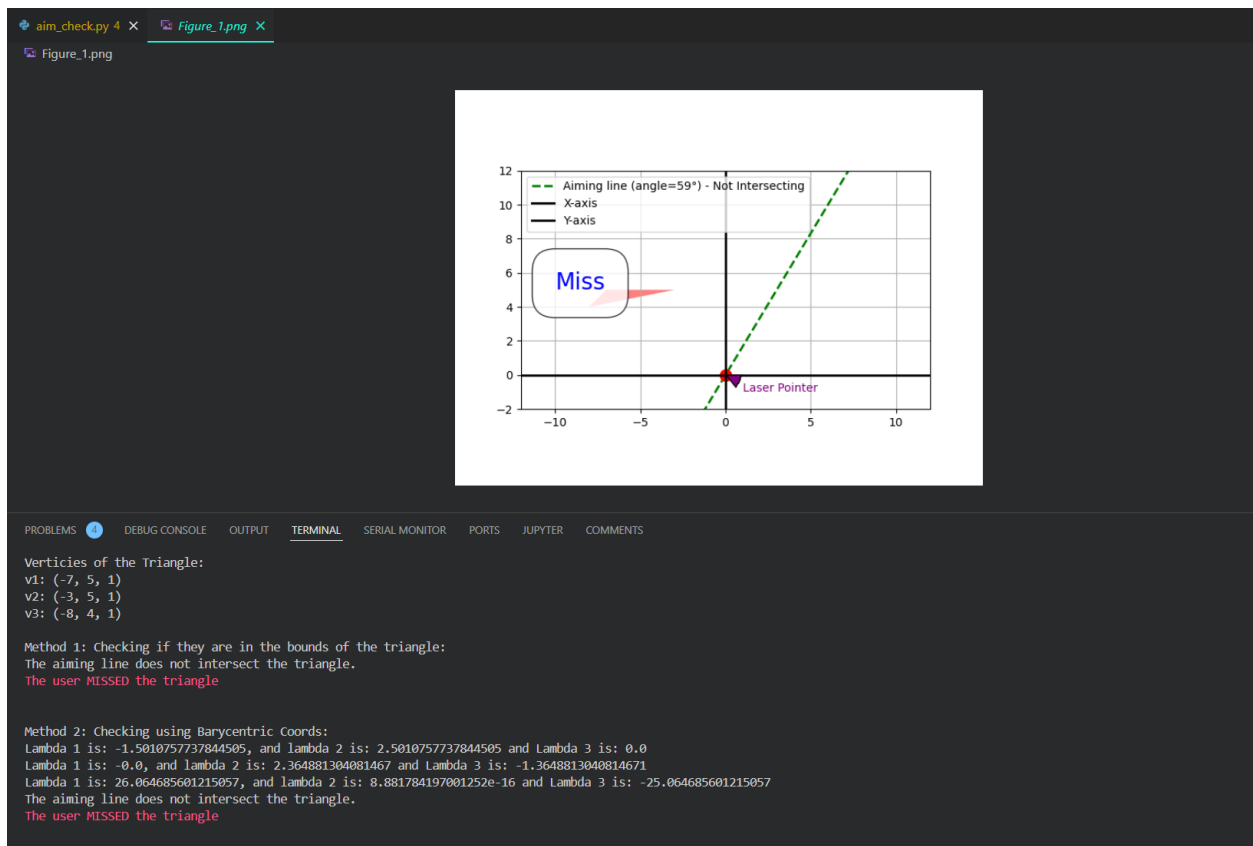


Figure 3: Figure 3

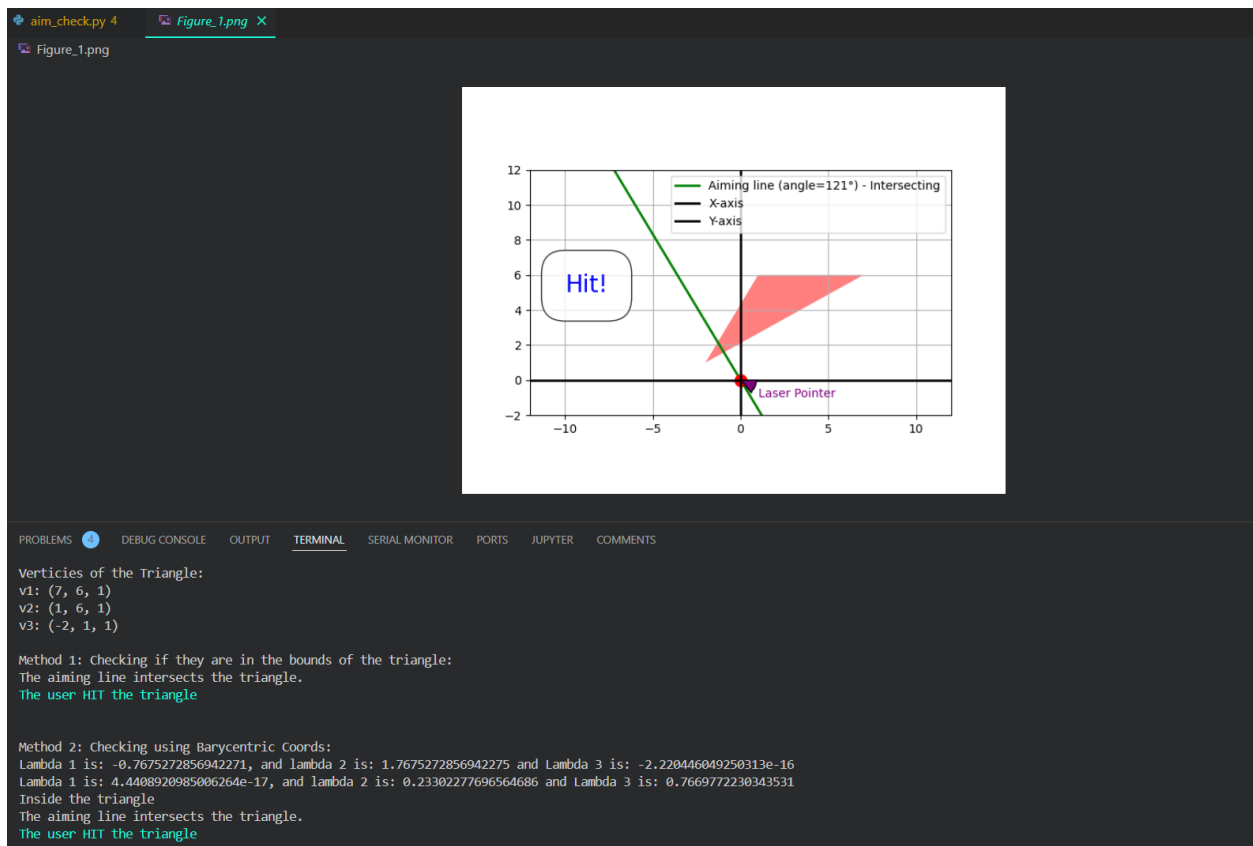


Figure 4: Figure 4

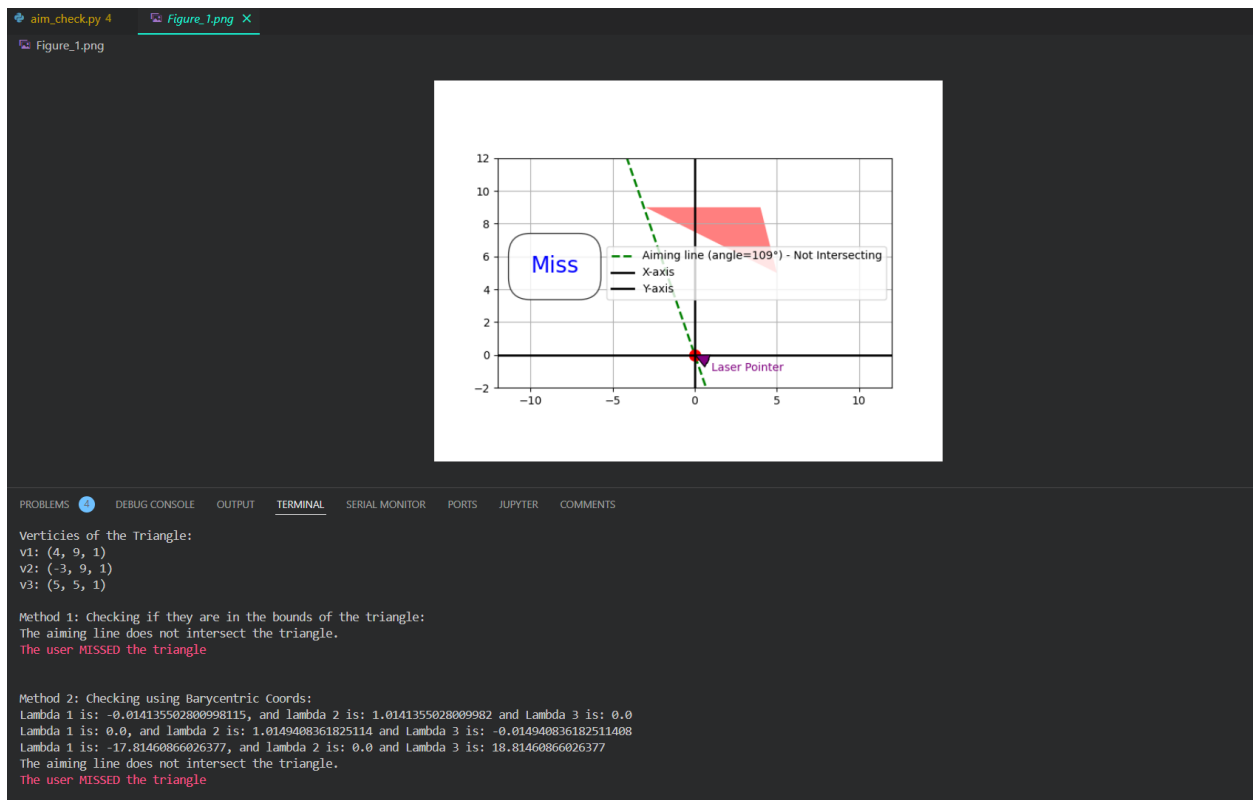


Figure 5: Figure 5

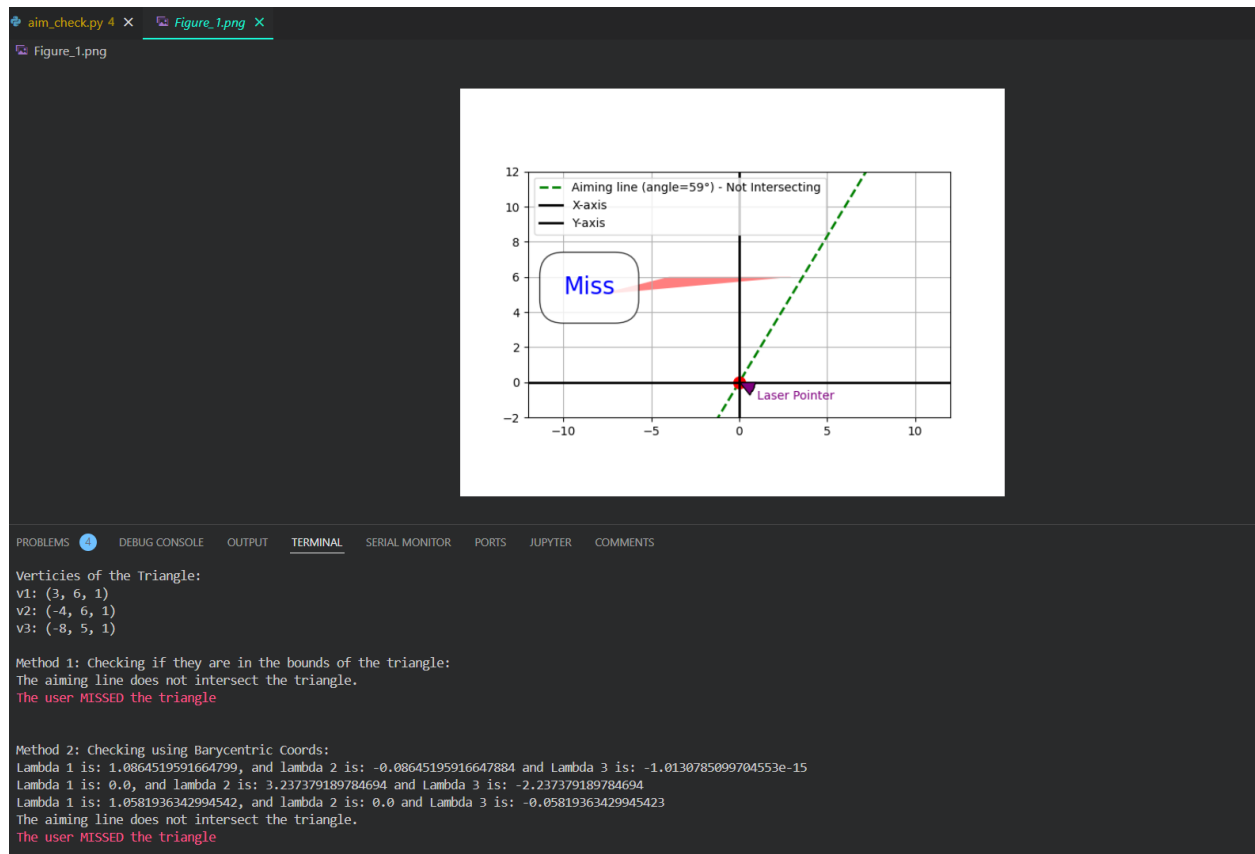


Figure 6: Figure 6

8.2 Code for the extra credit:

Code for the extra credit

```
1 import pandas as pd
2 import numpy as np
3 import random as rd
4 from typing import List, Tuple
5 import matplotlib.pyplot as plt
6 from termcolor import cprint
7
8 # Need a function for creating the plot and displaying the results: triangle
9 # filled red, user aim line is green!!!
10 # Need a function for creating random 3D points in repr. sapce -> maybe make
11 # 2D points then convert them
12 # Need a function for creating the bounds of the triangle. What ppr. does the
13 # triangle need to have ?
14 # Note: the user is always at the origin so that is set, change the angle as
15 # you go b/w 0 to 180 deg.
16 # Try and make the graph a 180 deg positive x-axis, this is the env.
17
18 overall_points = list()
19
20 '''
21 Denoting the vertices of the triangle as v1, v2, v3
22 v1, v2 will share thee same y co-ordinate for simplicity.
23 We are randomly choosing the length of the v1, v2 edge and the point v3
24 '''
25
26 def random_triangle() -> List[Tuple[int, int, int]]:
27     while True:
28         y_temp = rd.randint(5, 10) # Ensure y is always positive and above
29         the x-axis
30         v1 = (rd.randint(-8, 8), y_temp, 1)
31         v2 = (rd.randint(-8, 8), y_temp, 1)
32         v3 = (rd.randint(-8, 8), rd.randint(1, 5), 1) # Ensure v3's y-
33         coordinate is positive
34
35         # Make sure the triangle points down:
36         if v3[1] > y_temp:
37             continue
38
39         if not are_collinear(v1, v2, v3):
40             return [v1, v2, v3]
41
42 def are_collinear(v1: Tuple[int, int, int], v2: Tuple[int, int, int], v3:
43 Tuple[int, int, int]) -> bool:
44     # Calculate the determinant of the matrix formed by v1, v2, v3
45     mat = np.array([v1, v2, v3])
46     return np.linalg.det(mat) == 0
47
48 def compute_line(aiming_angle: float = rd.randint(0,180)) -> np.ndarray:
49
50     rad_angle = np.radians(aiming_angle)
51     return cross_product(np.array([np.cos(rad_angle), np.sin(rad_angle), 0]),
52 np.array([0, 0, 1])), aiming_angle
53
54 """Calculate the 3D cross product of two vectors in homogeneous coordinates.
55 """
56
57 def cross_product(v1: np.ndarray, v2: np.ndarray) -> np.ndarray:
```

```

47     return np.cross(v1, v2)
48
49     """Get the line equation in homogeneous coordinates from two points."""
50     def line_from_points(p1: Tuple[float, float, float], p2: Tuple[float, float,
51         float]) -> np.ndarray:
52         return cross_product(np.array(p1), np.array(p2))
53
54     """Determine if the aiming line intersects with any of the triangle's edges.
55     """
56     def find_intersection(vertices: List[Tuple[int, int, int]], line: np.ndarray)
57         -> bool:
58         for i in range(3):
59             # Get the current edge vertices
60             p1 = vertices[i]
61             p2 = vertices[(i + 1) % 3]
62
63             # Line equation for the edge
64             edge_line = line_from_points(p1, p2)
65
66             # Find the intersection point using cross product
67             intersection = cross_product(line, edge_line)
68             overall_points.append(intersection)
69
70             if intersection[2] != 0: # To avoid division by zero, then its a
71                 point at infinity
72                 intersection /= intersection[2] # Convert to Cartesian
73                 coordinates
74
75             # Check if the intersection point lies within the segment
76             if (min(p1[0], p2[0]) <= intersection[0] <= max(p1[0], p2[0]) and
77                 min(p1[1], p2[1]) <= intersection[1] <= max(p1[1], p2[1])):
78                 return True
79         return False
80
81     def plot_triangle_and_aim(vertices: List[Tuple[int, int, int]], line: np.
82         ndarray, aiming_angle: float, intersects: bool):
83         # Unpack vertices
84         x_values = [v[0] for v in vertices] + [vertices[0][0]]
85         y_values = [v[1] for v in vertices] + [vertices[0][1]]
86
87         # Plot triangle
88         plt.fill(x_values, y_values, 'red', alpha=0.5)
89
90         # Plot aim line
91         x = np.linspace(-12, 12, 100)
92         y = -(line[2] + line[0] * x) / line[1] # General form of line equation:
93             ax + by + c = 0
94
95         if intersects:
96             plt.plot(x, y, 'g-', linewidth=2, label=f'Aiming line (angle={
97                 aiming_angle} ) - Intersecting')
98         else:
99             plt.plot(x, y, 'g--', linewidth=2, label=f'Aiming line (angle={
100                 aiming_angle} ) - Not Intersecting')
101
102         # Origin point and direction arrow
103         plt.plot(0, 0, 'ro', markersize=10)
104         plt.annotate('Laser Pointer', xy=(0, 0), xytext=(1, -1),
105             arrowprops=dict(facecolor='purple', shrink=0.05),

```

```

97         fontsize=10, color='purple')
98
99     # Add "Hit" or "Miss" label with a larger box
100    hit_or_miss = 'Hit!' if intersects else 'Miss'
101    plt.text(-10, 5, f'{hit_or_miss}', fontsize=20, color='blue',
102            bbox=dict(facecolor='white', alpha=0.8, boxstyle="round,pad=1"))
103
104    # Plot settings
105    plt.xlim(-12, 12)
106    plt.ylim(-2, 12)
107    plt.axhline(0, color='black', linewidth=2, label='X-axis') # Blue X-axis
108    plt.axvline(0, color='black', linewidth=2, label='Y-axis') # Orange Y-
109                        axis
110    plt.grid(True)
111    plt.legend()
112    plt.gca().set_aspect('equal', adjustable='box')
113    plt.savefig("Figure_1.png")
114    plt.show()
115
116    # Method 2: barycentric coordinate
117    def calc_barycentric_coordinates(vertices: List[Tuple[int,int,int]], line: np
118    .ndarray) -> bool: #, points = List[Tuple[int, int,int]]
119        def normalize_point(point):
120            if point[2] != 0: # To avoid division by zero, then its a point at
121                            infinity
122                point /= point[2] # Convert to Cartesian coordinates
123            # print(f"This is the point: {point}") # Debug
124            return point
125
126        def find_points(vertices: List[Tuple[int, int, int]], line: np.ndarray)
127        -> List[Tuple[int,int,int]]:
128            points = []
129            for i in range(3):
130                # Get the current edge vertices
131                p1 = vertices[i]
132                p2 = vertices[(i + 1) % 3]
133
134                # Line equation for the edge
135                edge_line = line_from_points(p1, p2)
136
137                point = cross_product(line, edge_line)
138
139                normalize_point(point)
140
141                # Find the intersection point using cross product
142                points.append(point)
143            return points
144
145        def check_point_location(lambda1: float, lambda2: float, lambda3: float)
146        -> Tuple[str, bool]:
147            if lambda1 > 0 and lambda2 > 0 and lambda3 > 0:
148                return "Inside the triangle", True
149            elif (lambda1 == 0 and lambda2 > 0 and lambda3 > 0) or \
150                 (lambda2 == 0 and lambda1 > 0 and lambda3 > 0) or \
151                 (lambda3 == 0 and lambda1 > 0 and lambda2 > 0):
152                return "On an edge of the triangle", True
153            elif lambda1 < 0 or lambda2 < 0 or lambda3 < 0:
154                return "Outside the triangle", False

```



```

151         else:
152             return "Unknown condition" , False # This is a fallback, should
                not reached
153
154 points = find_points(vertices, line)
155
156 for point in points:
157     x_1, y_1 = vertices[0][0], vertices[0][1]
158     x_2, y_2 = vertices[1][0], vertices[1][1]
159     x_3, y_3 = vertices[2][0], vertices[2][1]
160     x, y = point[0], point[1]
161     lambda_1 = ((y_2 - y_3)*(x-x_3) + (x_3-x_2)*(y-y_3)) / ((y_2-y_3)*(
        x_1-x_3) + (x_3-x_2)*(y_1-y_3))
162     lambda_2 = ((y_3-y_1)*(x-x_3) + (x_1-x_3)*(y-y_3)) / ((y_2-y_3)*(x_1-
        x_3) + (x_3-x_2)*(y_1-y_3))
163     lambda_3 = 1 - lambda_1 - lambda_2
164
165     print(f"Lambda 1 is: {lambda_1}, and lambda 2 is: {lambda_2} and
        Lambda 3 is: {lambda_3}")
166     results = check_point_location(lambda_1, lambda_2, lambda_3)
167     if results[1]:
168         print(results[0])
169         return True
170     return False
171
172 if __name__ == '__main__':
173     vertices = random_triangle()
174     # vertices = [(3,5,1),(7,5,1),(5,3,1)]
175     print("Verticies of the Triangle:")
176     [print(f"v{i}: {vertices[i-1]}") for i in range(1, 4)] # Debug Statement
177
178     # Compute the aim line
179     l, aiming_angle = compute_line() # If you want to specify the angle put
        the angle(in deg) into the function
180
181     # Check if the aim line intersects the triangle
182     print()
183     print("Method 1: Checking if they are in the bounds of the triangle:")
184     if intersects := find_intersection(vertices, l):
185         print("The aiming line intersects the triangle.")
186         cprint("The user HIT the triangle", "green")
187     else:
188         print("The aiming line does not intersect the triangle.")
189         cprint("The user MISSED the triangle", "red")
190
191     print()
192     print()
193
194     print("Method 2: Checking using Barycentric Coords:")
195     if intersects := calc_barycentric_coordinates(vertices, l):
196         print("The aiming line intersects the triangle.")
197         cprint("The user HIT the triangle", "green")
198     else:
199         print("The aiming line does not intersect the triangle.")
200         cprint("The user MISSED the triangle", "red")
201
202     print()
203     print()
204

```

```
205 # Plot the triangle and the aim line  
206 plot_triangle_and_aim(vertices, l, aiming_angle, intersects)
```

Listing 1: aim.check.py